

API网关



版本：V1.0 2025.12.16 | 状态：已投产 | 核心技术：Apache APISIX + Nacos + Shell Automation

接口人：孙家正

核心设计理念

网关是 WisePen 平台的唯一流量入口，设计遵循“配置即代码 (IaC)”和“服务自治”原则。在运维管理上采用“脚本自动化为主，控制台可视化为辅”的混合模式。

- 统一入口**：收敛所有微服务的外部访问，统一处理 HTTPS、鉴权、限流和熔断。
- 双模管理**：
 - 生产/CI环境**：推荐使用 Shell 脚本 (IaC)，确保配置可版本化、可回溯、幂等执行。
 - 开发/调试环境**：允许使用 APISIX Dashboard（控制台）进行快速调整和即时查看。
- 动态发现**：深度集成 Nacos。网关不硬编码上游 IP，而是监听 Nacos 注册表，实现后端服务的自动扩缩容感知。
- 全链路可观测**：网关作为链路追踪 (Trace) 的起点，负责生成 TraceID 并透传给后端，打通“网关-应用-数据库”的监控闭环。

协议与契约

为了保证网关管理的有序性，所有接入方必须遵守以下基础设施契约。

端口协议

端口	协议	用途	访问权限
80	HTTP	流量主入口，前端 API 请求均发往此处	公网开放
9180	HTTP	Admin API，用于下发配置	仅限内网 (脚本使用)
9091	HTTP	Prometheus Metrics 指标暴露	仅限内网 (收集器使用)

路由 ID 规划契约

APISIX 中不同类型的资源 ID 命名空间是隔离的，但为了维护方便，我们要遵守以下约定：

Plugin Config ID (插件配置对象)

- ID **1**：[核心保留] 专用作 `Public Template (Monitor + Routing)` (全局公开模版)。**严禁覆盖或删除此 ID。**
- ID **2**：[核心保留] 专用作 `Protected Template (Monitor + Routing + JWT + AuthHeaders)` (全局鉴权模版)。**严禁覆盖或删除此 ID。**
- ID **3+**：开放给其他特殊策略使用。

Route ID (路由对象)

- ID **1-999**：预留给核心业务服务（如 User, Order 等），保持 ID 稳定。
- ID **1000+**：开发人员临时测试可随意使用。

请在新增路由时查/补路由表：[📖 路由表](#)

服务发现契约

- 网关只认 **Nacos** 中的服务名。
- 后端要求：后端 `bootstrap.yml` 中的 `spring.application.name` 必须与网关注册脚本中的 `service_name` 完全一致，否则报 503。

开发者接入指南

当开发一个新的微服务（例如 `wisepen-order-service`），需要将其加入网关才能被前端访问。我们提供两种方式将新服务接入网关，**推荐优先使用脚本**以保证环境一致性。

使用脚本自动化 (推荐)

我们封装了自动化脚本，**禁止手动调用 Admin API**。在服务器运行该脚本需要有开发服务器账号！没有的同学看[这里申请](#)：[📖 环境起步（二）服务器环境](#)

1. 打开项目中的 `gateway/setup_gateway.sh` 文件。
2. 找到底部的路由注册区域。
3. 添加一行注册命令：

代码块

```
1 # 语法: register_service_route <路由ID> <路由描述> <监听路径> <Nacos服务名>
2
3 # 示例: 注册订单服务
4 register_service_route 10 "order-service" "/order/*" "wisepen-order-service"
```

上述命令会自动绑定 ID=2 的插件模版，意味着该接口默认需要登录 (JWT 鉴权)。如果该服务是完全公开的（不需要登录认证），请使用 `register_public_route` 函数，会自动绑定 ID=1 的插件模版。

- 使用自动化脚本部署该脚本到服务器：

请在本地终端运行部署工具：

代码块

```
1  # 进入工具目录
2  cd deploy/dev-tools
3  # 运行交互式部署脚本
4  python remote_deploy.py gateway
```

脚本会自动打包本地 `gateway-apisix/remote_bundle` 目录，SCP 上传至服务器临时目录，自动修复 Windows 换行符 (CRLF)，并执行 `setup.sh`。如果看到 `[SUCCESS]`，说明路由已生效。

- 手动部署该脚本到服务器（不推荐）：

手动上传 `deploy/gateway-apisix/remote_bundle` 后，在服务器执行该文件夹下的 `setup.sh`。脚本执行完毕且无报错，即表示路由上线。

使用控制台可视化 (调试)

访问控制台：浏览器打开 `http://wisepen-dev-server:9000` (APISIX Dashboard)。

创建路由：

- 点击 路由 (Route) -> 创建。
- 路径: 填写 `/order/*`。
- 上游 (Upstream):
 - 类型选择：服务发现。
 - 发现类型：Nacos。
 - 服务名：`wisepen-order-service`。
- 插件配置 (Plugins):
 - 关键步骤：在“插件配置”步骤中，选择“绑定插件配置”。
 - 选择 ID 为 1 的 `WisePen Global Auth Template`。绑定模版后，该路由会自动继承鉴权、监控和 Lua 注入逻辑。如果该服务是完全公开的（不需要登录认证），请不要绑定该模板！

发布：点击提交。

核心配置解析

静态引导配置

位置: (开发服务器, 已映射至Docker内) `/opt/cloud-infra/apisix-conf/config.yaml`

作用: APISIX 启动时的元数据配置。此文件修改频率极低, 仅在基础设施变更时修改。

- 服务发现 (Discovery):

代码块

```
1  discovery:
2    nacos:
3      host: ["http://nacos:8848"] # 指向 Docker 容器名
4      prefix: "/nacos/v1/"
5      fetch_interval: 1           # 1秒拉取一次, 保证服务上下线秒级感知
```

- 可观测性 (Plugins):

代码块

```
1  plugins:
2    - prometheus      # 暴露指标
3    - opentelemetry   # 链路追踪核心
4  plugin_attr:
5    opentelemetry:
6      trace_id_source: x-request-id # 优先复用前端传来的ID
7      collector_address: otel-collector:4318 # 发送给 Collector
```

- Admin API: `allow_admin: 0.0.0.0/0` 允许了开发环境的脚本远程调用。

- 插件启用: 开启 `jwt-auth`, `serverless-pre-function`, `opentelemetry` 等核心插件。

动态初始化脚本

位置: WisePen核心项目目录: `deploy/gateway-apisix/remote_bundle/scripts/setup.sh`

作用: 业务逻辑的核心。它负责将我们定义的 Consumer、鉴权模版和路由规则推送到 APISIX 运行态。

该脚本包含三个关键动作:

1. Consumer 初始化: 创建 `wisepen_global_consumer`, 绑定密钥。

2. Global Template 初始化 (Plugin Config ID: 1): 定义标准鉴权套餐: `jwt-auth` (验签) + `auth.lua` (Header注入) + `opentelemetry` (监控)。

所有微服务路由只需 `plugin_config_id: 1` , 无需重复配置几十行插件参数。如果需要调整鉴权逻辑 (如修改 Lua) , 只需更新此模版, 所有绑定路由立即生效。

3. Route 初始化: 调用 Admin API 将 URL 映射到 Nacos 服务。

备注

- **幂等性:** `setup_gateway.sh` 脚本是幂等的。你可以 (也应该) 在每次修改配置后重复运行它, 它会覆盖旧配置, 不会产生副作用。
- **Lua 脚本:** `scripts/auth.lua` 是鉴权逻辑的核心。修改它后, 必须重新运行 `setup_gateway.sh` 才能生效 (因为脚本内容是作为字符串写入 Etcd 的) 。
- **静态配置生效:** 如果你修改了服务器上的 `config.yaml` , 必须重启 APISIX 容器 (`docker restart apisix`) 才能生效。
- **Etcd:** APISIX 的所有动态配置存储在 Etcd 中。如果删除了 Etcd 容器, 所有路由规则会丢失, 必须重新运行初始化脚本。

常见问题 (FAQ)

Q1: 前端请求报 404 Not Found?

A: 排查: 路由是否注册? 使用 `curl http://127.0.0.1:9180/apisix/admin/routes` 查看。Nacos 里有没有这个服务? 如果 Nacos 没服务, APISIX 找不到上游也会报 404 (或 503) 。

Q2: 前端请求报 502 Bad Gateway?

A: 网关从 Nacos 拿到了 IP, 但连不上。 **排查:** 检查后端微服务容器是否挂了, 或者 Docker 网络不通。

Q3: 为什么修改了 `config.yaml` 没生效?

A: 静态配置修改后, 必须重启 APISIX 容器 (`docker restart apisix`) 才能生效。而修改 `setup_gateway.sh` 只需要运行脚本即可, 无需重启容器。

Q4: 出现 `missing token` 或 `401` 错误?

A: 这是鉴权模版生效了。 **排查:** 检查请求头 `Authorization` 是否携带了 Bearer Token。如果确认带了还报 401, 检查 `setup_gateway.sh` 里的 `JWT_SECRET` 和 Java 端是否一致。